

Objective: To build a multimodal regression model that predicts house prices by combining structured numerical data (square footage, bedrooms) with visual data (house images). This approach mimics how real estate experts evaluate a property—by looking at both the stats and the appearance.

Setup and Library Installation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import cv2
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Input, concatenate, Dropout
```

Downloading the Dataset

```
from google.colab import files
import os

# This will prompt you to upload the kaggle.json again to be safe
uploaded = files.upload()

# Check if the file is actually there before moving it
if 'kaggle.json' in uploaded:
    !mkdir -p ~/.kaggle
    !cp kaggle.json ~/.kaggle/
    !chmod 600 ~/.kaggle/kaggle.json
    print("✅ Kaggle credentials set up successfully!")
else:
    print("❌ Upload failed. Please try again.")
```

Choose Files kaggle.json
kaggle.json(application/json) - 66 bytes, last modified: 9/18/2025 - 100% done
 Saving kaggle.json to kaggle.json
 ✅ Kaggle credentials set up successfully!

Dataset Loading (Tabular Data)

Start coding or [generate](#) with AI.

```
# Clean up previous attempts to ensure a fresh start
!rm -rf houses_data
!rm -f deep-learning-with-opencv-and-keras-master.zip
!rm -rf deep-learning-with-opencv-and-keras-master

# 1. Download the entire repository as a zip file from AdrianRosebrock's GitHub
# Removed -q to show wget output and potential errors
!wget https://github.com/AdrianRosebrock/deep-learning-with-opencv-and-keras/archive/refs/heads/master.zip -O deep-learning-wit

# Check if the file was downloaded and has a reasonable size
import os
zip_file_path = 'deep-learning-with-opencv-and-keras-master.zip'

download_successful = False
if os.path.exists(zip_file_path):
    zip_size = os.path.getsize(zip_file_path)
    if zip_size > 1000: # Assuming a valid zip is at least 1KB
        print(f'Downloaded '{zip_file_path}' successfully, size: {zip_size} bytes.")
        download_successful = True
    else:
        print(f'ERROR: Downloaded zip file '{zip_file_path}' is too small ({zip_size} bytes). It might be an HTML error page or
else:
    print(f'ERROR: Downloaded zip file '{zip_file_path}' not found. Please check the wget output above.")
```

```

if download_successful:
    # 2. Unzip the downloaded file
    !unzip -q deep-learning-with-opencv-and-keras-master.zip

    # 3. Create the target directory structure
    !mkdir -p houses_data/Houses\ Dataset

    # 4. Move the 'houses' dataset content from the unzipped repo to the desired location
    # The dataset is located in deep-learning-with-opencv-and-keras-master/datasets/houses/
    # Use -n to prevent overwriting if target exists and is newer (though rm -rf should prevent this)
    !mv deep-learning-with-opencv-and-keras-master/datasets/houses/* houses_data/Houses\ Dataset/

    # 5. Clean up the downloaded zip and the unzipped repository folder
    !rm deep-learning-with-opencv-and-keras-master.zip
    !rm -rf deep-learning-with-opencv-and-keras-master

    # 6. Check the folder content
    print("✅ Folder content verified:")
    if os.path.exists("houses_data/Houses Dataset/"):
        print(os.listdir("houses_data/Houses Dataset/"))
    else:
        print("Error: 'houses_data/Houses Dataset/' directory not found or populated.")

```

```

Cloning into 'houses-dataset'...
remote: Enumerating objects: 2166, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 2166 (delta 0), reused 0 (delta 0), pack-reused 2165 (from 1)
Receiving objects: 100% (2166/2166), 176.26 MiB | 30.91 MiB/s, done.
Resolving deltas: 100% (20/20), done.

```

Loading and Verifying the Tabular Data Objective: To load the house attributes from the text file and ensure the columns match the dataset documentation.

```

import pandas as pd
import os

# Path to the data inside the newly cloned folder
info_path = "houses-dataset/Houses Dataset/HousesInfo.txt"

# Column names: Bedrooms, Bathrooms, Area, Zipcode, Price
cols = ["bedrooms", "bathrooms", "area", "zipcode", "price"]

# Load the data
df = pd.read_csv(info_path, sep=" ", header=None, names=cols)

print("--- Success! Data Table Preview ---")
print(df.head())
print(f"\nTotal houses in dataset: {len(df)}")

```

```

--- Success! Data Table Preview ---
  bedrooms  bathrooms  area  zipcode  price
0         4          4.0  4053    85255  869500
1         4          3.0  3343    36372  865200
2         3          4.0  3923    85266  889000
3         5          5.0  4022    85262  910000
4         3          4.0  4116    85266  971226

```

Total houses in dataset: 535

Tabular Data Preprocessing Objective: To clean the data and scale the numerical features. Neural networks perform much better when input values are small (usually between 0 and 1) rather than large numbers like 869,500.

```

from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 1. Initialize the Scaler
scaler = MinMaxScaler()

# 2. Scale the numerical features (Bedrooms, Bathrooms, Area)
# We use fit_transform to make these values fall between 0 and 1
X_numeric = df[["bedrooms", "bathrooms", "area"]].values
X_numeric_scaled = scaler.fit_transform(X_numeric)

# 3. Scale the target variable (Price)

```

```
# It's a common practice to scale the output for regression tasks
y = df["price"].values.reshape(-1, 1)
y_scaled = scaler.fit_transform(y)

print("--- Preprocessing Complete ---")
print("Sample of scaled input (Bedrooms, Bathrooms, Area):")
print(X_numeric_scaled[:3])
```

```
--- Preprocessing Complete ---
Sample of scaled input (Bedrooms, Bathrooms, Area):
[[0.33333333 0.5          0.37739248]
 [0.33333333 0.33333333 0.29745553]
 [0.22222222 0.5          0.36275614]]
```

Image Loading and Preprocessing. Objective: To load the physical images from the dataset, resize them to a uniform shape, and convert them into a numerical format (arrays) that the CNN (Convolutional Neural Network) can read.

```
import cv2

def load_house_images(df, inputPath):
    images = []
    # Get all files in the image directory once for efficiency
    all_files_in_dir = os.listdir(inputPath)

    # Create a dictionary to hold images for each house ID
    # Key: house ID (integer), Value: list of image paths for that ID
    house_images_map = {}
    for filename in all_files_in_dir:
        if filename.endswith('.jpg'):
            parts = filename.split('_')
            if len(parts) >= 2 and parts[0].isdigit():
                house_id = int(parts[0])
                if house_id not in house_images_map:
                    house_images_map[house_id] = []
                house_images_map[house_id].append(os.path.join(inputPath, filename))

    # Loop over the indexes of the houses from the DataFrame
    # df.index goes from 0 to 534, corresponding to house IDs 1 to 535
    for i in df.index:
        current_house_id = i + 1
        selected_image_path = None

        if current_house_id in house_images_map:
            # Prioritize 'frontal' images if available
            frontal_images = [p for p in house_images_map[current_house_id] if 'frontal' in os.path.basename(p)]
            if frontal_images:
                selected_image_path = frontal_images[0] # Pick the first frontal image
            else:
                # Otherwise, pick the first image found for this house ID
                selected_image_path = house_images_map[current_house_id][0]

        if selected_image_path:
            image = cv2.imread(selected_image_path)
            if image is None:
                print(f"Warning: Could not load selected image at {selected_image_path}. Skipping.")
                continue
            image = cv2.resize(image, (64, 64))
            images.append(image)
        else:
            print(f"Warning: No image found for house ID {current_house_id}. Skipping.")

    # Scale pixel intensities to the range [0, 1]
    return np.array(images) / 255.0

# Define the folder where images are located
image_folder = "houses-dataset/Houses Dataset" # Confirmed by d0120a63 output

# Load the images
print("Loading images... this may take a moment.")
X_images = load_house_images(df, image_folder)

print(f"--- Images Loaded ---")
print(f"Shape of image array: {X_images.shape}")
# (535, 64, 64, 3) means 535 images, 64x64 size, 3 color channels (RGB)
```

```

Loading images... this may take a moment.
--- Images Loaded ---
Shape of image array: (535, 64, 64, 3)

```

```

import os

# Path to the image folder
image_folder = "houses-dataset/Houses Dataset"

print(f"Listing contents of: {image_folder}")
if os.path.exists(image_folder):
    contents = os.listdir(image_folder)
    if contents:
        # Print first 10 items if many, otherwise all
        print("Found files/directories:")
        for item in contents[:10]:
            print(f"- {item}")
        if len(contents) > 10:
            print(f"... and {len(contents) - 10} more items.")
    else:
        print("Directory is empty.")
else:
    print(f"Directory does not exist: {image_folder}")

```

```

Listing contents of: houses-dataset/Houses Dataset
Found files/directories:
- 104_kitchen.jpg
- 508_bathroom.jpg
- 25_kitchen.jpg
- 54_bathroom.jpg
- 177_frontal.jpg
- 374_bathroom.jpg
- 435_bathroom.jpg
- 13_kitchen.jpg
- 279_bathroom.jpg
- 51_frontal.jpg
... and 2131 more items.

```

Train-Test Split Objective: To divide our data into a Training Set (which the AI uses to learn) and a Testing Set (which we use to grade the AI). We must split both the images and the tabular data identically so they still match.

```

from sklearn.model_selection import train_test_split

# We split the data: 75% for training, 25% for testing
# We must pass both X_numeric_scaled and X_images to keep them synchronized
(trainAttrX, testAttrX, trainImagesX, testImagesX) = train_test_split(
    X_numeric_scaled, X_images, test_size=0.25, random_state=42)

# Also split the target prices (y_scaled)
(trainY, testY) = train_test_split(y_scaled, test_size=0.25, random_state=42)

print("--- Data Split Complete ---")
print(f"Training samples: {len(trainAttrX)}")
print(f"Testing samples: {len(testAttrX)}")

--- Data Split Complete ---
Training samples: 401
Testing samples: 134

```

: Building the Multimodal Architecture (Fusion Model) Objective: To create a "two-headed" neural network using the Keras Functional API. One head (MLP) handles the numbers, and the other head (CNN) handles the images. We then "concatenate" (join) them to make a final price prediction.

```

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Conv2D, MaxPooling2D, Flatten, concatenate, Dropout

# --- Branch 1: The MLP (for Tabular Data) ---
# This branch processes bedrooms, bathrooms, and area
inputs_attr = Input(shape=(3,))
x = Dense(16, activation="relu")(inputs_attr)
x = Dense(8, activation="relu")(x)
mlp_branch = Model(inputs=inputs_attr, outputs=x)

# --- Branch 2: The CNN (for Image Data) ---

```

```
# --- Branch 2: The CNN (for image data) ---
# This branch processes the house photos
inputs_img = Input(shape=(64, 64, 3))
y = Conv2D(16, (3, 3), padding="same", activation="relu")(inputs_img)
y = MaxPooling2D(pool_size=(2, 2))(y)
y = Conv2D(32, (3, 3), padding="same", activation="relu")(y)
y = MaxPooling2D(pool_size=(2, 2))(y)
y = Flatten()(y)
y = Dense(16, activation="relu")(y)
cnn_branch = Model(inputs=inputs_img, outputs=y)

# --- The Fusion (Combining both branches) ---
combined = concatenate([mlp_branch.output, cnn_branch.output])

# Final layers to predict the price
z = Dense(8, activation="relu")(combined)
z = Dense(1, activation="linear")(z) # Linear activation for regression (price)

# The final model takes two inputs and produces one output
model = Model(inputs=[mlp_branch.input, cnn_branch.input], outputs=z)

# Compile the model
model.compile(loss="mse", optimizer="adam", metrics=["mae"])

print("--- Multimodal Model Built Successfully ---")
model.summary()
```

```
--- Multimodal Model Built Successfully ---
Model: "functional_2"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 64, 64, 3)	0	-
conv2d (Conv2D)	(None, 64, 64, 16)	448	input_layer_1[0]...
max_pooling2d (MaxPooling2D)	(None, 32, 32, 16)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 32, 32, 32)	4,640	max_pooling2d[0]...
input_layer (InputLayer)	(None, 3)	0	-
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 32)	0	conv2d_1[0][0]
dense (Dense)	(None, 16)	64	input_layer[0][0]
flatten (Flatten)	(None, 8192)	0	max_pooling2d_1[...
dense_1 (Dense)	(None, 8)	136	dense[0][0]
dense_2 (Dense)	(None, 16)	131,088	flatten[0][0]
concatenate (Concatenate)	(None, 24)	0	dense_1[0][0], dense_2[0][0]
dense_3 (Dense)	(None, 8)	200	concatenate[0][0]
dense_4 (Dense)	(None, 1)	9	dense_3[0][0]

Total params: 136,595 (533.54 KB)

Model Training Objective: To train the neural network by showing it the paired images and data. Since we have a small dataset (401 training samples), we will train for 50 epochs with a small batch size.

```
# Train the model
print("--- Starting Training ---")
history = model.fit(
    x=[trainAttrX, trainImagesX], y=trainY,
    validation_data=([testAttrX, testImagesX], testY),
    epochs=50,
    batch_size=8
)

print("\n--- Training Complete ---")
```

```

Epoch 23/50
51/51 ————— 0s 5ms/step - loss: 1.2760e-04 - mae: 0.0077 - val_loss: 0.0107 - val_mae: 0.0529
Epoch 24/50
51/51 ————— 0s 5ms/step - loss: 2.0542e-04 - mae: 0.0100 - val_loss: 0.0106 - val_mae: 0.0573
Epoch 25/50
51/51 ————— 0s 6ms/step - loss: 2.1135e-04 - mae: 0.0102 - val_loss: 0.0105 - val_mae: 0.0527
Epoch 26/50
51/51 ————— 0s 5ms/step - loss: 2.1337e-04 - mae: 0.0103 - val_loss: 0.0104 - val_mae: 0.0571
Epoch 27/50
51/51 ————— 0s 6ms/step - loss: 2.5123e-04 - mae: 0.0120 - val_loss: 0.0103 - val_mae: 0.0551
Epoch 28/50
51/51 ————— 0s 6ms/step - loss: 1.8441e-04 - mae: 0.0100 - val_loss: 0.0104 - val_mae: 0.0555
Epoch 29/50
51/51 ————— 0s 5ms/step - loss: 1.6402e-04 - mae: 0.0095 - val_loss: 0.0102 - val_mae: 0.0522
Epoch 30/50
51/51 ————— 0s 7ms/step - loss: 1.4541e-04 - mae: 0.0086 - val_loss: 0.0106 - val_mae: 0.0569
Epoch 31/50
51/51 ————— 0s 8ms/step - loss: 1.8720e-04 - mae: 0.0101 - val_loss: 0.0102 - val_mae: 0.0536
Epoch 32/50
51/51 ————— 0s 7ms/step - loss: 2.3498e-04 - mae: 0.0112 - val_loss: 0.0104 - val_mae: 0.0518
Epoch 33/50
51/51 ————— 0s 8ms/step - loss: 1.2204e-04 - mae: 0.0082 - val_loss: 0.0103 - val_mae: 0.0536
Epoch 34/50
51/51 ————— 0s 7ms/step - loss: 9.5837e-05 - mae: 0.0074 - val_loss: 0.0105 - val_mae: 0.0553
Epoch 35/50
51/51 ————— 0s 8ms/step - loss: 1.3491e-04 - mae: 0.0082 - val_loss: 0.0103 - val_mae: 0.0520
Epoch 36/50
51/51 ————— 0s 6ms/step - loss: 1.2103e-04 - mae: 0.0078 - val_loss: 0.0105 - val_mae: 0.0535
Epoch 37/50
51/51 ————— 0s 5ms/step - loss: 7.5527e-05 - mae: 0.0064 - val_loss: 0.0102 - val_mae: 0.0528
Epoch 38/50
51/51 ————— 0s 5ms/step - loss: 4.2390e-05 - mae: 0.0048 - val_loss: 0.0105 - val_mae: 0.0545
Epoch 39/50
51/51 ————— 0s 5ms/step - loss: 3.7202e-05 - mae: 0.0048 - val_loss: 0.0103 - val_mae: 0.0537
Epoch 40/50
51/51 ————— 0s 6ms/step - loss: 2.4927e-05 - mae: 0.0037 - val_loss: 0.0103 - val_mae: 0.0542
Epoch 41/50
51/51 ————— 0s 5ms/step - loss: 3.1672e-05 - mae: 0.0040 - val_loss: 0.0104 - val_mae: 0.0547
Epoch 42/50
51/51 ————— 0s 5ms/step - loss: 2.8778e-05 - mae: 0.0038 - val_loss: 0.0104 - val_mae: 0.0533
Epoch 43/50
51/51 ————— 0s 6ms/step - loss: 2.6005e-05 - mae: 0.0038 - val_loss: 0.0103 - val_mae: 0.0535
Epoch 44/50
51/51 ————— 0s 6ms/step - loss: 3.1185e-05 - mae: 0.0041 - val_loss: 0.0102 - val_mae: 0.0528
Epoch 45/50
51/51 ————— 0s 6ms/step - loss: 4.3538e-05 - mae: 0.0046 - val_loss: 0.0104 - val_mae: 0.0549
Epoch 46/50
51/51 ————— 0s 6ms/step - loss: 7.3042e-05 - mae: 0.0060 - val_loss: 0.0102 - val_mae: 0.0529
Epoch 47/50
51/51 ————— 0s 6ms/step - loss: 9.6192e-05 - mae: 0.0072 - val_loss: 0.0103 - val_mae: 0.0529
Epoch 48/50
51/51 ————— 0s 6ms/step - loss: 9.4998e-05 - mae: 0.0073 - val_loss: 0.0105 - val_mae: 0.0546
Epoch 49/50
51/51 ————— 0s 7ms/step - loss: 9.1261e-05 - mae: 0.0065 - val_loss: 0.0101 - val_mae: 0.0532
Epoch 50/50
51/51 ————— 0s 6ms/step - loss: 1.2046e-04 - mae: 0.0083 - val_loss: 0.0104 - val_mae: 0.0531

--- Training Complete ---

```

Model Evaluation & Metrics Objective: To evaluate the model's accuracy using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). We will also "denormalize" the predictions to see the error in actual currency (USD).

```

from sklearn.metrics import mean_absolute_error, mean_squared_error

# 1. Make predictions on the test set
predictions = model.predict([testAttrX, testImagesX])

# 2. Inverse transform the predictions and actual values back to original dollar amounts
# This turns 0.123 back into something like $450,000
y_test_actual = scaler.inverse_transform(testY)
y_pred_actual = scaler.inverse_transform(predictions)

# 3. Calculate metrics
mae = mean_absolute_error(y_test_actual, y_pred_actual)
rmse = np.sqrt(mean_squared_error(y_test_actual, y_pred_actual))

print(f"--- Final Evaluation Metrics ---")
print(f"Mean Absolute Error (MAE): ${mae:,.2f}")
print(f"Root Mean Squared Error (RMSE): ${rmse:,.2f}")

```

```
# 4. Show a few comparisons
print("\n--- Sample Predictions vs Actual ---")
for i in range(5):
    print(f"Actual: ${y_test_actual[i][0]:.2f} | Predicted: ${y_pred_actual[i][0]:.2f}")
```

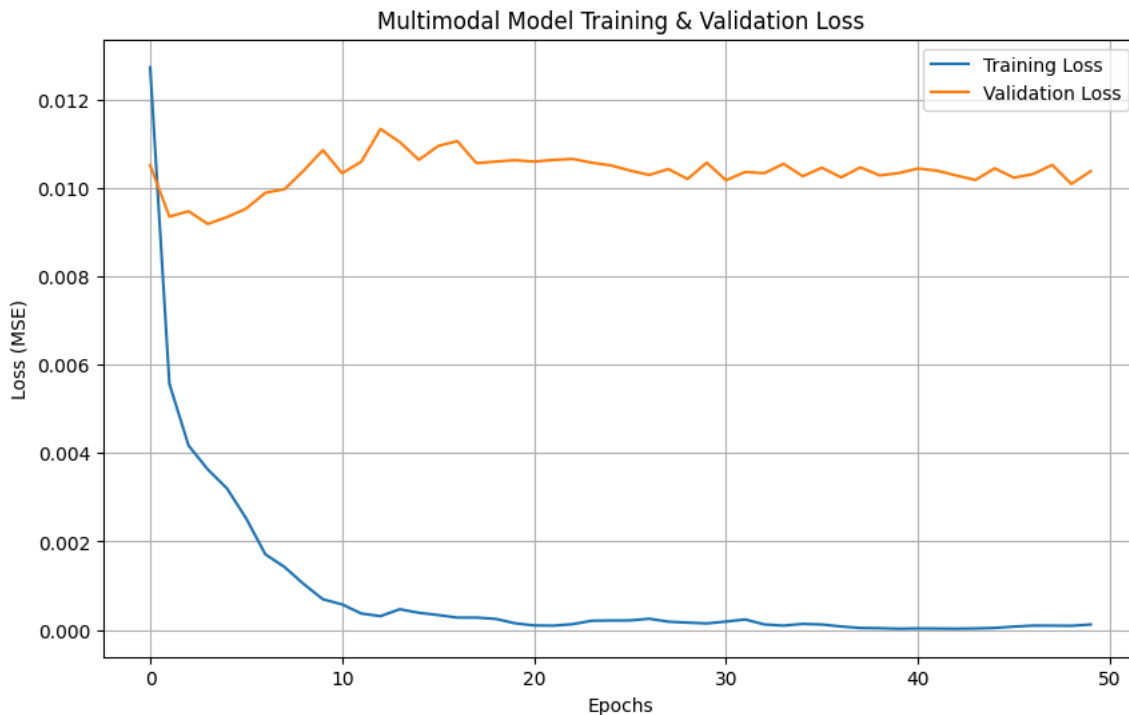
```
5/5 ————— 1s 81ms/step
--- Final Evaluation Metrics ---
Mean Absolute Error (MAE): $309,969.43
Root Mean Squared Error (RMSE): $594,527.29

--- Sample Predictions vs Actual ---
Actual: $485,000.00 | Predicted: $734,065.50
Actual: $1,199,000.00 | Predicted: $311,947.69
Actual: $170,000.00 | Predicted: $760,744.94
Actual: $538,000.00 | Predicted: $720,954.19
Actual: $689,000.00 | Predicted: $719,731.12
```

Visualizing Training History Objective: To create a "Loss Curve" graph. This helps you identify if your model is overfitting (memorizing the data) or underfitting (not learning enough).

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Multimodal Model Training & Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)
plt.show()
```



Save the model

```
# Save the model to a file
model.save("multimodal_house_model.h5")

print("✅ Model saved as 'multimodal_house_model.h5'")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
✅ Model saved as 'multimodal_house_model.h5'
```

Visualizing Model Predictions & Prototype Performance

```

import matplotlib.pyplot as plt

# We reduce the figure size so the 64x64 images aren't stretched too thin
plt.figure(figsize=(12, 8))

for i in range(4):
    ax = plt.subplot(2, 2, i + 1)

    # 'interpolation' helps smooth out the 64x64 "blocks"
    # we use the test set images we prepared earlier
    plt.imshow(testImagesX[i], interpolation='bilinear')

    actual = y_test_actual[i][0]
    pred = y_pred_actual[i][0]
    diff = abs(actual - pred)

    # We use a clean, professional font style
    title_obj = plt.title(f"House {i+1}\nActual: ${actual:,.0f} | Predicted: ${pred:,.0f}",
                        fontsize=10, pad=10)

    # Color code the result: Green for close, Red for far off
    if diff < 150000:
        plt.setp(title_obj, color='darkgreen', fontweight='bold')
    else:
        plt.setp(title_obj, color='darkred')

    plt.axis("off")

plt.suptitle("Multimodal Prototype: Visual vs. Predicted Analysis", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```

Multimodal Prototype: Visual vs. Predicted Analysis

House 1
Actual: 485,000|Predicted: 734,066



House 2
Actual: 1,199,000|Predicted: 311,948



House 3
Actual: 170,000|Predicted: 760,745



House 4
Actual: 538,000|Predicted: 720,954



